

Avaliação - Banco de Dados, API e Windows Forms

07/10/2026

Cenário: sistema Fruteira (cadastro e estoque de frutas)

Professor(a):	Valor: 10,0	Nota:
---------------	-------------	-------

Instruções

- A prova tem **20 questões** de múltipla escolha, cada uma com cinco alternativas (A a E) e **uma única** correta. Cada questão vale 0,5 ponto.
- As questões se apoiam nos **Quadros 1, 2 e 3** das páginas seguintes. Leia os quadros antes de responder: a resposta certa depende do que está escrito neles, e não de como um sistema parecido "costuma" funcionar.
- Quando a questão disser "o que acontece", acompanhe o código linha por linha, com os dados do Quadro 1.
- Marque as respostas no **cartão-resposta** da última página. Só o cartão é corrigido.

Quadro 1 — Banco de dados (MySQL, motor InnoDB)

```
CREATE TABLE categoria (  
  id INT NOT NULL AUTO_INCREMENT,  
  descricao VARCHAR(50) NOT NULL,  
  PRIMARY KEY (id)  
);  
  
CREATE TABLE fruta (  
  id INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(80) NOT NULL,  
  preco DECIMAL(6,2) NOT NULL,  
  quantidade INT NOT NULL,  
  id_categoria INT NOT NULL,  
  data_validade DATETIME NOT NULL,  
  hash_img LONGTEXT NULL,  
  PRIMARY KEY (id),  
  CONSTRAINT fk_fruta_categoria  
    FOREIGN KEY (id_categoria) REFERENCES categoria (id)  
);
```

Dados gravados (considere sempre estes dados como ponto de partida de cada questão):

Tabela categoria

id	descricao
1	Cítrica
2	Tropical
3	Vermelha
4	Seca

Tabela fruta (a coluna hash_img foi omitida)

id	nome	preco	quantidade	id_categoria	data_validade
1	Laranja Pera	4.90	80	1	2026-12-10
2	Banana Prata	7.49	120	2	2026-11-20
3	Morango	12.00	35	3	2026-10-30
4	Abacaxi	9.90	15	2	2026-11-05
5	Limão Taiti	3.50	60	1	2026-12-01

Quadro 2 — API (ASP.NET Core, Controllers + Entity Framework Core)

A entidade Fruta da API está mapeada para a tabela do Quadro 1, e o JSON usa os mesmos nomes das colunas (id, nome, preco, quantidade, id_categoria, data_validade, hash_img). A API responde em <https://localhost:7069>.

```
[ApiController]  
[Route("[controller]")]  
public class FrutaController : ControllerBase  
{  
  private readonly FruteiraContext _context;  
  
  public FrutaController(FruteiraContext context)  
  {  
    _context = context;  
  }  
  
  [HttpGet]  
  public IActionResult Listar([FromQuery] int id_categoria, [FromQuery] string? nome)  
  {  
    var consulta = _context.Frutas.AsQueryable();  
  
    if (id_categoria > 0)
```

```

        consulta = consulta.Where(f => f.IdCategoria == id_categoria);

        if (!string.IsNullOrEmpty(nome))
            consulta = consulta.Where(f => f.Nome.Contains(nome));

        return Ok(consulta.ToList());
    }

    [HttpPost]
    public IActionResult Cadastrar([FromBody] Fruta fruta)
    {
        _context.Frutas.Add(fruta);
        _context.SaveChanges();
        return Created("/Fruta/" + fruta.Id, fruta);
    }

    [HttpPut("{id}")]
    public IActionResult Atualizar(int id, [FromBody] Fruta fruta)
    {
        var existente = _context.Frutas.Find(id);
        if (existente == null)
            return NotFound();

        existente.Nome = fruta.Nome;
        existente.Preco = fruta.Preco;
        existente.Quantidade = fruta.Quantidade;
        existente.IdCategoria = fruta.IdCategoria;
        existente.DataValidade = fruta.DataValidade;
        _context.SaveChanges();
        return NoContent();
    }
}

```

Quadro 3 — Programa Windows Forms (cliente da API)

Trecho A — FrutaApi: categorias. ListarCategorias() faz GET /Categoria e devolve uma lista **vazia** quando a API falha. Fruta.Tipos é a lista reserva { "Cítrica", "Tropical", "Vermelha", "Comum", "Seca" }, que tem só os textos.

```

private static string[] tiposCarregados; // null = ainda não carregou
private static List<Categoria> categoriasCarregadas = new List<Categoria>();

public static string[] ObterTipos()
{
    if (tiposCarregados != null)
        return tiposCarregados;

    List<string> descricoes = new List<string>();
    List<Categoria> categorias = new List<Categoria>();

    foreach (Categoria categoria in ListarCategorias())
    {
        descricoes.Add(categoria.Descricao);
        categorias.Add(categoria);
    }

    categoriasCarregadas = categorias;

    if (descricoes.Count == 0)
        tiposCarregados = Fruta.Tipos;
    else
        tiposCarregados = descricoes.ToArray();

    return tiposCarregados;
}

public static int ObterIdCategoria(string descricao)
{
    ObterTipos();
    foreach (Categoria categoria in categoriasCarregadas)

```

```

        if (categoria.Descricao == descricao)
            return categoria.Id;
        return 0;
    }

    public static string ObterDescricaoCategoria(int idCategoria)
    {
        ObterTipos();
        foreach (Categoria categoria in categoriasCarregadas)
            if (categoria.Id == idCategoria)
                return categoria.Descricao;
        return string.Empty;
    }

```

Trecho B — FrutaApi: pesquisar e cadastrar. UrlFrutas vale "https://localhost:7069/Fruta".

```

public static List<Fruta> Pesquisar(int idCategoria, string nome)
{
    List<string> filtros = new List<string>();

    if (idCategoria > 0)
        filtros.Add("id_categoria=" + idCategoria);

    if (!string.IsNullOrEmpty(nome))
        filtros.Add("nome=" + Uri.EscapeDataString(nome));

    string url = UrlFrutas;
    if (filtros.Count > 0)
        url = url + "?" + string.Join("&", filtros);

    try
    {
        // GetAsync(url); se o status não for de sucesso devolve lista vazia;
        // senão converte o JSON em List<Fruta> e devolve.
    }
    catch (Exception)
    {
        return new List<Fruta>();
    }
}

public static bool Cadastrar(Fruta fruta)
{
    try
    {
        // ... converte a fruta em JSON (variável corpo) ...
        HttpResponseMessage resposta = cliente.PostAsync(UrlFrutas, corpo).Result;
        return resposta.IsSuccessStatusCode;
    }
    catch (Exception)
    {
        return false;
    }
}

```

Trecho C — EstoqueForm. As colunas do IstEstoque (modo View.Details) estão nesta ordem: Fruta | Quantidade | Validade | Valor | Tipo. A constante TodasAsCategorias vale "(Todas)".

```

private void EstoqueForm_Load(object sender, EventArgs e)
{
    cmbCategoria.Items.Add(TodasAsCategorias);
    cmbCategoria.Items.AddRange(FrutaApi.ObterTipos());
    cmbCategoria.SelectedIndex = 0;
}

private void btnPesquisar_Click(object sender, EventArgs e)
{
    int idCategoria = 0;

    string categoria = cmbCategoria.SelectedItem.ToString();
    if (categoria != TodasAsCategorias)
        idCategoria = FrutaApi.ObterIdCategoria(categoria);
}

```

```

string nome = txtNome.Text.Trim();

List<Fruta> encontradas = FrutaApi.Pesquisar(idCategoria, nome);
PreencherLista(encontradas);
}

private void PreencherLista(List<Fruta> frutas)
{
    lstEstoque.Items.Clear();

    if (frutas == null || frutas.Count == 0)
    {
        MessageBox.Show("Nenhuma fruta encontrada com estes filtros.");
        return;
    }

    foreach (Fruta fruta in frutas)
    {
        ListViewItem linha = new ListViewItem(fruta.Nome);
        linha.SubItems.Add(fruta.Quantidade.ToString());
        linha.SubItems.Add(fruta.DataValidade.ToShortDateString());
        linha.SubItems.Add(fruta.Preco.ToString("C"));
        linha.SubItems.Add(FrutaApi.ObterDescricaoCategoria(fruta.IdCategoria));
        lstEstoque.Items.Add(linha);
    }
}

private void btnVoltar_Click(object sender, EventArgs e)
{
    this.Close();
}

```

Trecho D — CadastroForm. O método Avisar(mensagem, controle) mostra a mensagem em um MessageBox e devolve o foco para o controle informado.

```

private void CadastroForm_Load(object sender, EventArgs e)
{
    cmbTipo.Items.AddRange(FrutaApi.ObterTipos());
}

private void btnCadastrar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtNome.Text))
    { Avisar("Informe o nome da fruta.", txtNome); return; }

    int quantidade;
    if (!int.TryParse(txtQuantidade.Text, out quantidade))
    { Avisar("A quantidade deve ser um número inteiro.", txtQuantidade); return; }

    if (quantidade <= 0)
    { Avisar("A quantidade deve ser maior que zero.", txtQuantidade); return; }

    if (cmbTipo.SelectedIndex < 0)
    { Avisar("Escolha o tipo da fruta.", cmbTipo); return; }

    int idCategoria = FrutaApi.ObterIdCategoria(cmbTipo.SelectedItem.ToString());
    if (idCategoria == 0)
    { Avisar("Não foi possível obter as categorias da API.", cmbTipo); return; }

    decimal valor;
    if (!decimal.TryParse(txtValor.Text, out valor))
    { Avisar("O valor deve ser um número.", txtValor); return; }

    if (valor <= 0)
    { Avisar("O valor deve ser maior que zero.", txtValor); return; }

    if (string.IsNullOrEmpty(caminhoImagem))
    { Avisar("Escolha a imagem da fruta.", btnEscolherImagem); return; }
}

```

```
// ... monta o objeto fruta com os dados da tela ...

if (!FrutaApi.Cadastrar(fruta))
{
    MessageBox.Show("Não foi possível cadastrar a fruta. Tente novamente.");
    return;
}

MessageBox.Show("Fruta cadastrada com sucesso!");
LimparCampos();
AbrirEstoque();
}

private void AbrirEstoque()
{
    using (EstoqueForm estoque = new EstoqueForm())
    {
        this.Hide();
        estoque.ShowDialog();
    }

    this.Show();
}
```

SQL — chave estrangeira e tipos de dados

Questão 1 (0,5) — Partindo dos dados do Quadro 1, os três comandos abaixo são executados **um de cada vez**, nesta ordem, com o autocommit ligado (padrão do MySQL).

```
INSERT INTO fruta (nome, preco, quantidade, id_categoria, data_validade)
VALUES ('Uva Passa', 18.00, 10, 4, '2027-01-15');
```

```
INSERT INTO fruta (nome, preco, quantidade, id_categoria, data_validade)
VALUES ('Kiwi', 6.00, 25, 5, '2026-11-30');
```

```
INSERT INTO fruta (nome, preco, quantidade, id_categoria, data_validade)
VALUES ('Figo', 8.00, 12, 4, '2026-11-12');
```

Ao final, qual é o resultado de `SELECT COUNT(*) FROM fruta`; e por quê?

- A) 8. O Kiwi é gravado com `id_categoria` igual a `NULL`, porque a categoria 5 não existe.
- B) 5. O erro do segundo comando desfaz também o cadastro da Uva Passa.
- C) 7. Apenas o Kiwi é recusado, porque não existe categoria com `id` 5; os outros dois comandos são independentes e ficam gravados.
- D) 6. Depois do erro de chave estrangeira a tabela fica bloqueada e o Figo não é gravado.
- E) 8. O MySQL cria sozinho a categoria 5 para manter a integridade referencial.

Questão 2 (0,5) — A chave estrangeira do Quadro 1 foi criada **sem** a cláusula `ON DELETE`. Considere os comandos abaixo, cada um executado **isoladamente** sobre os dados originais:

- I. `DELETE FROM categoria WHERE id = 4;`
- II. `DELETE FROM categoria WHERE id = 2;`
- III. `DELETE FROM fruta WHERE id_categoria = 2;`

Quais comandos são executados **com sucesso**?

- A) Apenas I e III.
- B) Apenas I.
- C) Apenas III.
- D) I, II e III.
- E) Apenas I e II.

Questão 3 (0,5) — Um aluno quer que, ao apagar uma categoria, as frutas dela fiquem "sem categoria". Mantendo a definição das colunas **exatamente** como no Quadro 1, ele apaga a FK antiga e executa:

```
ALTER TABLE fruta
ADD CONSTRAINT fk_fruta_categoria
FOREIGN KEY (id_categoria) REFERENCES categoria (id)
ON DELETE SET NULL;
```

O que acontece?

- A) A FK é criada e, ao apagar a categoria 2, Banana Prata e Abacaxi ficam com `id_categoria` igual a `NULL`.
- B) O `ALTER TABLE` é recusado: `id_categoria` foi declarada `NOT NULL`, e `ON DELETE SET NULL` só pode ser usado em coluna que aceite `NULL`.
- C) A FK é criada normalmente; o erro só aparece depois, quando alguém tentar apagar uma categoria que tenha frutas.
- D) A FK é criada e, ao apagar a categoria 2, as frutas dela ficam com `id_categoria` igual a 0.
- E) A FK é criada e, ao apagar a categoria 2, as frutas dela são apagadas junto.

Questão 4 (0,5) — A coluna `preco` é `DECIMAL(6,2)`. Com o MySQL no modo padrão (`sql_mode` estrito), são executados, um de cada vez:

```
UPDATE fruta SET preco = 7.495 WHERE id = 2; -- Banana Prata
```

```
UPDATE fruta SET preco = 12500.00 WHERE id = 3; -- Morango
```

Quais preços ficam gravados para Banana Prata e Morango?

- A) Banana Prata: 7.49 — Morango: 9999.99
- B) Banana Prata: 7.495 — Morango: 12500.00
- C) Banana Prata: 7.50 — Morango: 9999.99
- D) Banana Prata: 7.49 — Morango: 12.00
- E) Banana Prata: 7.50 — Morango: 12.00

Questão 5 (0,5) — Sobre os tipos de dados escolhidos no Quadro 1, analise as afirmativas:

I. Trocar preco de DECIMAL(6,2) para FLOAT não mudaria nenhum resultado, porque os dois tipos guardam as casas decimais de forma exata.

II. hash_img é LONGTEXT porque a imagem viaja em Base64: o texto de uma foto de 2 MB não cabe em um VARCHAR nem em um TEXT, que param em 65.535 bytes.

III. Se fruta.id_categoria fosse declarada como VARCHAR(10), o MySQL recusaria a criação da FK, porque a coluna precisa ter tipo compatível com categoria.id.

IV. Como data_validade é DATETIME, uma validade gravada como '2026-12-31' fica armazenada como 2026-12-31 00:00:00.

Estão corretas:

- A) Apenas I e IV.
- B) Apenas II e III.
- C) Apenas III e IV.
- D) Apenas II, III e IV.
- E) I, II, III e IV.

Questão 6 (0,5) — Considere duas situações, com a verificação de chaves estrangeiras ligada (padrão):

Situação 1: em um banco **vazio**, o aluno executa primeiro o CREATE TABLE fruta do Quadro 1 e só depois o CREATE TABLE categoria.

Situação 2: em um banco que já tem as duas tabelas do Quadro 1, o aluno executa DROP TABLE categoria;.

O que acontece em cada situação?

- A) As duas funcionam: o MySQL só confere a chave estrangeira quando uma linha é inserida.
- B) A situação 1 falha; a situação 2 funciona e apaga também as frutas, em cascata.
- C) A situação 1 funciona, porque a FK só é validada no INSERT; a situação 2 falha.
- D) As duas falham: a tabela referenciada precisa existir antes de quem a referencia, e não pode ser apagada enquanto uma FK apontar para ela.
- E) A situação 1 falha; a situação 2 funciona e a FK de fruta passa a apontar para uma tabela inexistente.

API — rotas GET, POST e PUT

Questão 7 (0,5) — As categorias foram carregadas da API (Quadro 1). Na tela de Estoque o usuário escolhe **Tropical** e digita no campo nome o texto " banana prata " (com dois espaços antes e um depois). Ao clicar em Pesquisar, qual endereço o programa chama? (Quadro 3, trechos A, B e C)

- A) `https://localhost:7069/Fruta?id_categoria=2&nome=banana%20prata`
- B) `https://localhost:7069/Fruta/2?nome=banana%20prata`
- C) `https://localhost:7069/Fruta?id_categoria=Tropical&nome=banana%20prata`
- D) `https://localhost:7069/Fruta?id_categoria=2&nome=%20%20banana%20prata%20`
- E) `https://localhost:7069/Fruta?id_categoria=2&nome=banana prata`

Questão 8 (0,5) — Na tela de Estoque o usuário deixa o filtro em **(Todas)**, não digita nada no nome e clica em Pesquisar. Com a API do Quadro 2 no ar e os dados do Quadro 1, o que acontece?

- A) O programa não chama a API quando não há filtro e mostra direto "Nenhuma fruta encontrada".
- B) O programa chama GET /Fruta sem query string; na API id_categoria chega valendo 0 e nome chega null, nenhum Where é aplicado e a resposta é 200 com as 5 frutas.
- C) A API responde 400 (Bad Request), porque id_categoria é um parâmetro obrigatório.
- D) O programa envia ?id_categoria=0&nome= e a API devolve uma lista vazia, pois não existe categoria 0.
- E) A API responde 404 (Not Found), porque a rota /Fruta só existe com filtros.

Questão 9 (0,5) — É preciso acrescentar ao FrutaController uma rota que atenda GET /Fruta/3, devolvendo a fruta de id 3 ou 404 quando ela não existir. O corpo do método é sempre o mesmo, usando o parâmetro recebido:

```
var fruta = _context.Frutas.Find(<parâmetro>);  
if (fruta == null) return NotFound();  
return Ok(fruta);
```

Qual par atributo + assinatura faz a rota funcionar como pedido?

- A) `[HttpGet] public IActionResult Buscar([FromBody] int id)`
- B) `[HttpGet("id")] public IActionResult Buscar(int id)`
- C) `[HttpPost("{id}")] public IActionResult Buscar(int id)`
- D) `[HttpGet("{id}")] public IActionResult Buscar(int codigo)`
- E) `[HttpGet("{id}")] public IActionResult Buscar(int id)`

Questão 10 (0,5) — Com os dados do Quadro 1 e o controller do Quadro 2, quais frutas são devolvidas pela chamada abaixo?

```
GET https://localhost:7069/Fruta?id_categoria=1&nome=an
```

- A) Laranja Pera, Banana Prata e Morango.
- B) Laranja Pera e Limão Taiti.
- C) Apenas Laranja Pera.
- D) Laranja Pera, Limão Taiti, Banana Prata e Morango.
- E) Nenhuma, porque Contains exige que o nome seja exatamente igual ao texto informado.

Questão 11 (0,5) — Suponha que FrutaApi.Cadastrar (Quadro 3, trecho B) seja chamado por btnCadastrar_Click com uma fruta cujo id_categoria vale 9. O controller do Quadro 2 não tem try/catch. O que acontece na API e na tela?

- A) A API responde 201 e a fruta é gravada com id_categoria 9; a tela mostra "Fruta cadastrada com sucesso!".
- B) A API responde 400 automaticamente, porque [ApiController] confere as chaves estrangeiras antes de entrar no método.
- C) O SaveChanges() falha por causa da FK e a API responde 500; no programa, Cadastrar devolve false, aparece "Não foi possível cadastrar a fruta", os campos continuam preenchidos e o Estoque não abre.
- D) A API responde 500; o HttpClient lança uma exceção ao receber esse código e o programa fecha.
- E) A API responde 404, pois a categoria não foi encontrada; a tela limpa os campos e abre o Estoque.

Questão 12 (0,5) — Com os dados do Quadro 1, a requisição abaixo é enviada ao controller do Quadro 2. Antes dela, o hash_img da fruta 3 valia "ANTIGA".

```
PUT https://localhost:7069/Fruta/3
Content-Type: application/json
```

```
{ "id": 5, "nome": "Morango Orgânico", "preco": 15.00, "quantidade": 35,
  "id_categoria": 3, "data_validade": "2026-10-30T00:00:00", "hash_img": "NOVA" }
```

Qual é o resultado?

- A) A linha de id 5 (Limão Taiti) é alterada, porque o id enviado no corpo tem prioridade sobre o da rota.
- B) A linha de id 3 é alterada por completo, inclusive hash_img = "NOVA", e a resposta é 200 com a fruta atualizada.
- C) A API responde 400, porque o id da rota (3) é diferente do id do corpo (5).
- D) A linha de id 3 passa a ser "Morango Orgânico" por 15.00, mas o hash_img continua "ANTIGA"; a linha 5 não muda e a resposta é 204, sem corpo.
- E) Uma nova fruta é criada com id 6, porque o PUT insere quando o id do corpo não confere.

Questão 13 (0,5) — Ao digitar o controller, um aluno esqueceu o "{id}" e o método ficou assim (o resto é idêntico ao Quadro 2):

```
[HttpPut]
public IActionResult Atualizar(int id, [FromBody] Fruta fruta)
```

O que acontece ao chamar PUT https://localhost:7069/Fruta/3 com um JSON válido no corpo?

- A) Funciona igual: o ASP.NET Core descobre o id sozinho pelo final do endereço.
- B) A API responde 404: não existe mais rota de PUT para /Fruta/3. O método passou a atender PUT /Fruta, e o id teria de vir na query string (/Fruta?id=3).
- C) A API responde 405 (Method Not Allowed), porque /Fruta/3 só aceita GET.
- D) O projeto da API não compila, porque todo [HttpPut] exige um parâmetro de rota.
- E) A API responde 400, e depois de corrigido o corpo a fruta 3 é atualizada normalmente.

Windows Forms

Questão 14 (0,5) — No método AbrirEstoque (Quadro 3, trecho D), um aluno troca `estoque.ShowDialog();` por `estoque.Show();`, sem mexer em mais nada. O que o usuário vê ao clicar no botão que abre o Estoque?

- A) Nada muda: `Show()` e `ShowDialog()` só diferem no nome.
- B) As duas telas ficam abertas lado a lado e o usuário pode usar qualquer uma.
- C) O programa trava, porque duas telas não podem estar visíveis ao mesmo tempo.
- D) O Cadastro fica escondido para sempre e o programa continua rodando sem nenhuma tela.
- E) O Estoque aparece e some quase na mesma hora: `Show()` não espera a tela fechar, o `using` termina e descarta o Estoque, e `this.Show()` traz o Cadastro de volta.

Questão 15 (0,5) — Em `PreencherLista` (Quadro 3, trecho C), um aluno troca de lugar as duas linhas abaixo, deixando a do preço **antes** da data de validade:

```
linha.SubItems.Add(fruta.Preco.ToString("C"));  
linha.SubItems.Add(fruta.DataValidade.ToShortDateString());
```

O que acontece ao pesquisar?

- A) O programa roda sem erro, mas os valores aparecem trocados: o preço fica sob o cabeçalho "Validade" e a data fica sob "Valor".
- B) O projeto não compila, porque a ordem dos `SubItems` precisa ser igual à das colunas.
- C) O programa lança uma exceção ao tentar colocar um valor em dinheiro em uma coluna de data.
- D) Os cabeçalhos "Validade" e "Valor" também trocam de lugar, e a tabela continua correta.
- E) O `ListView` reconhece o formato de cada texto e coloca cada valor na coluna certa.

Questão 16 (0,5) — Em um formulário, a ligação entre o botão e o seu método fica no arquivo `CadastroForm.Designer.cs`, na linha `this.btnCadastrar.Click += new System.EventHandler(this.btnCadastrar_Click);`. Analise as afirmativas:

- I. Se essa linha for apagada e o método `btnCadastrar_Click` continuar em `CadastroForm.cs`, o projeto compila, mas clicar no botão não faz nada.
- II. Se o método `btnCadastrar_Click` for apagado de `CadastroForm.cs` e essa linha continuar no Designer, o projeto não compila.
- III. Trocar apenas o texto da linha `this.btnCadastrar.Name = "btnCadastrar";` por outro nome faz o clique parar de funcionar.
- IV. O código de `CadastroForm_Load` roda de novo toda vez que o Cadastro reaparece com `this.Show()`, depois de o Estoque ser fechado.

Estão corretas:

- A) Apenas I.
- B) Apenas I e II.
- C) Apenas II e III.
- D) Apenas I, II e IV.
- E) I, II, III e IV.

Programação

Questão 17 (0,5) — As categorias foram carregadas da API. Na tela de Cadastro o usuário preenche Nome = Kiwi, Quantidade = 10, **não escolhe** nenhum Tipo, digita Valor = abc e não escolhe imagem. Ao clicar em Cadastrar (Quadro 3, trecho D), o que acontece?

- A) Aparecem três mensagens em sequência: a do tipo, a do valor e a da imagem.
- B) Aparece somente "O valor deve ser um número.", porque texto inválido é verificado antes dos outros campos.
- C) Aparece somente "Escolha o tipo da fruta." e o foco vai para o cmbTipo; as demais verificações não chegam a ser executadas.
- D) O programa quebra com `NullReferenceException` em `cmbTipo.SelectedItem.ToString()`.
- E) A fruta é enviada para a API com `id_categoria` igual a 0 e a API recusa o cadastro.

Questão 18 (0,5) — Em um computador configurado em português do Brasil, considere o código:

```
int quantidade;  
bool ok = int.TryParse("12,5", out quantidade);
```

Quais são os valores de `ok` e `quantidade`, e o que aconteceria se, no lugar do `TryParse`, fosse usado `Convert.ToInt32("12,5")`?

- A) `ok = true` e `quantidade = 12`; o `Convert.ToInt32` também devolveria 12.
- B) `ok = true` e `quantidade = 13`; o `Convert.ToInt32` também arredondaria para 13.
- C) O código não compila, porque `quantidade` é usada sem ter recebido um valor inicial.
- D) `ok = false` e `quantidade = 0`; o `Convert.ToInt32` lançaria uma `FormatException`.
- E) `ok = false` e `quantidade = 0`; o `Convert.ToInt32` devolveria 0, sem erro.

Questão 19 (0,5) — O método abaixo, da tela de Estoque, divide a largura da tabela entre as cinco colunas. Todas as variáveis e propriedades envolvidas são `int`.

```
int largura = lstEstoque.ClientSize.Width;  
  
colFruta.Width    = largura * 30 / 100;  
colQuantidade.Width = largura * 15 / 100;  
colValidade.Width  = largura * 16 / 100;  
colValor.Width     = largura * 15 / 100;  
colTipo.Width      = largura * 24 / 100;
```

Se `largura` vale **790**, qual é a soma das larguras das cinco colunas?

- A) 788
- B) 790
- C) 791
- D) 789
- E) 787

Questão 20 (0,5) — O programa foi aberto com a API **desligada**: o CadastroForm_Load rodou e chamou ObterTipos(). Em seguida, **sem fechar o programa**, a API foi ligada (com os dados do Quadro 1). O usuário abre o Estoque, escolhe **Tropical**, deixa o nome em branco e clica em Pesquisar. Seguindo o código do Quadro 3 (trechos A, B e C), o que aparece na tabela?

- A) Apenas Banana Prata e Abacaxi, com "Tropical" na coluna Tipo.
- B) Nenhuma linha: aparece a mensagem "Nenhuma fruta encontrada com estes filtros."
- C) As 5 frutas, cada uma com o Tipo correto, porque ObterTipos() busca as categorias de novo ao perceber que a API voltou.
- D) O programa quebra, porque ObterIdCategoria devolve null quando não encontra a categoria.
- E) As 5 frutas, todas com a coluna Tipo em branco.

Cartão-resposta

Aluno(a): _____ Turma: _____

Escreva, em letra de forma, a letra da alternativa escolhida em cada questão. Rasuras anulam a questão.

Questão	1	2	3	4	5	6	7	8	9	10
Resposta										

Questão	11	12	13	14	15	16	17	18	19	20
Resposta										